Tên học phần: **ĐIỆN TOÁN ĐÁM MÂY**
Số tín chỉ: **3**
Giảng viên: **TS. Lê Quốc Tuấn**

ĐỀ TÀI ĐỒ ÁN MÔN HỌC
ĐIỆN TOÁN ĐÁM MÂY

Dành cho nhóm 3 sinh viên

Tài liệu định hướng nghiên cứu nâng cao và thực nghiệm

Biên soạn từ nội dung Bài giảng Điện toán đám mây gồm 5 chương

Thành phố Hồ Chí Minh, 2026

Hướng dẫn sử dụng

Tài liệu cung cấp 30 đề tài đồ án có mức độ nghiên cứu và thực nghiệm phù hợp với sinh viên ngành công nghệ thông tin. Mỗi đề tài yêu cầu nhóm chuyển một vấn đề kiến trúc hoặc vận hành thành câu hỏi có thể kiểm chứng, triển khai nguyên mẫu, thu thập dữ liệu đo và bảo vệ kết luận bằng minh chứng. Giảng viên có thể điều chỉnh quy mô hạ tầng nhưng nên giữ nguyên câu hỏi nghiên cứu, các chỉ số cốt lõi và yêu cầu tái lập để bảo đảm đồ án không dừng ở việc cài đặt công cụ.

Yêu cầu chung đối với mọi nhóm

- Nhóm gồm 3 sinh viên; mỗi thành viên phụ trách chính một mảng nhưng phải hiểu toàn bộ kiến trúc, mã nguồn và kết quả đo.
- Thời gian gợi ý 6-7 tuần. Phạm vi tối thiểu phải chạy được trên máy cá nhân, phòng máy hoặc gói miễn phí; mọi tài nguyên có phí phải đặt ngân sách và cảnh báo.
- Sử dụng dữ liệu công khai, dữ liệu tổng hợp hoặc dữ liệu đã ẩn danh. Không đưa mật khẩu, khóa API, dữ liệu cá nhân hay thông tin nội bộ vào kho mã nguồn và ảnh minh chứng.
- Kho sản phẩm phải chứa README, sơ đồ kiến trúc, mã nguồn, tệp cấu hình/IaC, kịch bản kiểm thử, dữ liệu đo thô, hướng dẫn tái lập và danh sách phiên bản công nghệ.
- Báo cáo phải tách rõ giả thuyết, thiết kế thí nghiệm, kết quả quan sát, diễn giải và giới hạn. Mọi biểu đồ phải có tên trục, đơn vị, điều kiện đo và số lần lặp.
- Video hoặc buổi demo cần chứng minh cả luồng bình thường và ít nhất một tình huống lỗi; ảnh chụp giao diện không thay thế cho log, số liệu hoặc kịch bản có thể chạy lại.

Sản phẩm bàn giao tối thiểu

- Báo cáo kỹ thuật 20-35 trang A4, không tính phụ lục mã nguồn và ảnh log dài.
- Kho mã nguồn có lịch sử đóng góp của 3 thành viên và thẻ phiên bản cho bản nghiệm thu.
- Sơ đồ kiến trúc hiện trạng/mục tiêu, sơ đồ luồng dữ liệu hoặc trình tự khi đề tài yêu cầu.
- Bộ dữ liệu đo dạng CSV/JSON và notebook hoặc script tạo lại bảng, biểu đồ.
- Hướng dẫn triển khai, dọn tài nguyên và ước tính chi phí; danh mục rủi ro bảo mật còn lại.
- Video demo 8-12 phút hoặc trình diễn trực tiếp 15-20 phút, có kịch bản và phân công rõ.

Gợi ý phân công trong nhóm

Vai trò chính	Trách nhiệm trọng tâm	Yêu cầu phối hợp
Sinh viên 1	Kiến trúc, hạ tầng, IaC và triển khai.	Cùng kiểm tra bảo mật và giải thích số liệu.
Sinh viên 2	Ứng dụng, dữ liệu, tích hợp và tự động hóa.	Cùng vận hành thí nghiệm và xử lý lỗi.
Sinh viên 3	Kiểm thử, quan sát, đo lường và tổng hợp báo cáo.	Cùng review mã và chứng minh khả năng tái lập.

Danh mục 30 đề tài môn học

Mã số	Đề tài	Chương
1	Đánh giá và thiết kế lại kiến trúc website thương mại điện tử theo khung Well Architected	Chương 1
2	Phân tích độ sẵn sàng của kiến trúc nhiều vùng sẵn sàng và kiểm chứng cơ chế chuyển đổi dự phòng	Chương 1
3	Tối ưu tự động co giãn cho dịch vụ web theo tải và theo chỉ số nghiệp vụ	Chương 1
4	Thiết kế kết nối hybrid cloud và đánh giá tính di động của một dịch vụ giữa hai môi trường	Chương 1
5	Xây dựng mô hình lựa chọn dịch vụ đám mây dựa trên SLA TCO và rủi ro	Chương 1
6	Xây dựng đám mây IaaS quy mô phòng thí nghiệm bằng nền tảng ảo hóa mã nguồn mở	Chương 2
7	Triển khai kiến trúc IaaS ba tầng với mạng phân đoạn và cổng truy cập quản trị an toàn	Chương 2
8	Phát triển ứng dụng PaaS có cơ sở dữ liệu được quản lý và đánh giá gánh nặng vận hành	Chương 2
9	Xây dựng hệ thống xử lý ảnh hướng sự kiện bằng serverless và đánh giá cold start	Chương 2
10	Xây dựng nền tảng CaaS nhỏ cho nhiều nhóm phát triển với quota và cô lập tài nguyên	Chương 2
11	So sánh thực nghiệm IaaS PaaS và FaaS cho cùng một dịch vụ API	Chương 2
12	Đánh giá block file và object storage theo mẫu truy cập ứng dụng	Chương 3
13	Thiết kế cơ sở dữ liệu quan hệ sẵn sàng cao với sao lưu phục hồi đến thời điểm	Chương 3
14	Thiết kế mô hình dữ liệu NoSQL và kiểm chứng đánh đổi CAP cho hệ thống danh mục sản phẩm	Chương 3
15	Xây dựng data lakehouse cho dữ liệu bán hàng bằng quy trình ETL và ELT	Chương 3
16	Xây dựng pipeline xử lý dữ liệu thời gian thực và đánh giá độ trễ đầu cuối	Chương 3
17	Xây dựng danh mục dữ liệu và truy vết dòng dữ liệu cho môi trường phân tích đám mây	Chương 3
18	Thiết kế IAM theo nguyên tắc quyền tối thiểu và kiểm chứng bằng nhật ký kiểm toán	Chương 4

Mã số	Đề tài	Chương
19	Xây dựng mô hình Zero Trust cho truy cập ứng dụng cloud native	Chương 4
20	Bảo vệ dữ liệu bằng mã hóa quản lý khóa và vòng đời bí mật	Chương 4
21	Xây dựng đường ống DevSecOps phát hiện cấu hình sai và lỗ hổng trước triển khai	Chương 4
22	Xây dựng giám sát tập trung và diễn tập ứng phó sự cố trên đám mây	Chương 4
23	Thiết kế và kiểm chứng kế hoạch sao lưu phục hồi thảm họa cho ứng dụng cloud native	Chương 4
24	Hiện đại hóa ứng dụng nguyên khối thành kiến trúc cloud native có đo kiểm	Chương 5
25	Bảo vệ chuỗi cung ứng container bằng SBOM ký image và xác minh chính sách	Chương 5
26	Triển khai ứng dụng trên Kubernetes với tự phục hồi co giãn và cập nhật không gián đoạn	Chương 5
27	Xây dựng CI CD với chiến lược blue green và canary có tự động rollback	Chương 5
28	Quản lý hạ tầng bằng mã và GitOps với phát hiện sai lệch cấu hình	Chương 5
29	Xây dựng khả năng quan sát và SRE gắn với tối ưu chi phí cho ứng dụng microservices	Chương 5
30	Triển khai dịch vụ suy luận AI trên đám mây với tự động co giãn và giám sát	Chương 5

Nhóm 1 Nền tảng và kiến trúc điện toán đám mây

Phạm vi bám sát Chương 1. Các đề tài tập trung vào yêu cầu phi chức năng, kiến trúc tham chiếu, co giãn, sẵn sàng, hybrid cloud và quyết định dựa trên chi phí - rủi ro.

Đề tài 1 Đánh giá và thiết kế lại kiến trúc website thương mại điện tử theo khung Well Architected

Câu hỏi nghiên cứu. Một website ba tầng đang chạy trên một máy chủ duy nhất cần thay đổi như thế nào để đáp ứng đồng thời yêu cầu sẵn sàng, bảo mật, hiệu năng và kiểm soát chi phí?

Mục tiêu của đề tài

- Phân tích yêu cầu nghiệp vụ và chuyển chúng thành yêu cầu kiến trúc có thể kiểm chứng, gồm lưu lượng dự kiến, thời gian đáp ứng, mức sẵn sàng và ngân sách.
- Thiết kế kiến trúc mục tiêu dựa trên các trụ cột của một khung Well-Architected và giải thích các đánh đổi thay vì chỉ liệt kê dịch vụ.
- Xây dựng nguyên mẫu hoạt động của luồng người dùng chính và đo sự cải thiện so với kiến trúc ban đầu.

Nội dung lý thuyết cần báo cáo

- Năm đặc trưng của điện toán đám mây; mô hình dịch vụ và mô hình triển khai.
- Kiến trúc tham chiếu, vùng địa lý, vùng sẵn sàng, cân bằng tải, co giãn và dịch vụ được quản lý.
- Các trụ cột về độ tin cậy, bảo mật, hiệu quả hiệu năng, tối ưu chi phí và vận hành.
- Phương pháp truy vết từ yêu cầu nghiệp vụ đến quyết định kiến trúc; phân tích điểm lỗi đơn.

Yêu cầu thực nghiệm

- Dựng phiên bản cơ sở gồm ứng dụng web và cơ sở dữ liệu trên một nút; ghi nhận độ trễ, thông lượng và hành vi khi dịch vụ dừng.
- Triển khai phiên bản cải tiến có bộ cân bằng tải, ít nhất hai bản sao ứng dụng, cơ sở dữ liệu tách riêng, lưu trữ đối tượng và giám sát.
- Tạo tải theo ba mức bình thường, cao điểm và tăng đột biến; chủ động dừng một bản sao ứng dụng để quan sát khả năng phục hồi.
- Ước tính chi phí tháng cho hai kiến trúc theo cùng một tập giả định và thực hiện phân tích nhạy cảm khi lưu lượng tăng gấp 2 và 5 lần.

Kết quả và minh chứng cần đạt

- Hai sơ đồ kiến trúc có chú giải luồng dữ liệu, ranh giới tin cậy và trách nhiệm quản trị.
- Bảng yêu cầu phi chức năng kèm chỉ số chấp nhận; dữ liệu đo p50, p95, thông lượng, tỷ lệ lỗi và thời gian phục hồi.
- Ma trận đánh giá theo từng trụ cột, nêu rõ rủi ro còn lại và quyết định không triển khai trong phạm vi đồ án.
- Mã nguồn, cấu hình triển khai, kịch bản tạo tải, dữ liệu đo thô và video minh chứng lỗi - phục hồi.

Tài liệu nghiên cứu gợi ý. [1], [2], [6]-[8] và Bài giảng Chương 1.

Công nghệ gợi ý. Docker Compose hoặc Kubernetes; Nginx/HAProxy; PostgreSQL; MinIO; Prometheus, Grafana; k6 hoặc JMeter; có thể triển khai trên AWS, Azure, Google Cloud hay môi trường cục bộ tương đương.

Đề tài 2 Phân tích độ sẵn sàng của kiến trúc nhiều vùng sẵn sàng và kiểm chứng cơ chế chuyển đổi dự phòng

Câu hỏi nghiên cứu. Việc nhân bản dịch vụ sang nhiều vùng sẵn sàng cải thiện độ sẵn sàng đến mức nào, và giả định độc lập giữa các thành phần có phù hợp với kết quả thực nghiệm không?

Mục tiêu của đề tài

- Tính toán độ sẵn sàng của kiến trúc nối tiếp, song song và có dự phòng bằng mô hình xác suất đơn giản.
- Thiết kế một dịch vụ chịu được lỗi của một nút ứng dụng và một nút dữ liệu trong phạm vi đã định.
- So sánh độ sẵn sàng lý thuyết với số liệu quan sát từ các lần tiêm lỗi có kiểm soát.

Nội dung lý thuyết cần báo cáo

- SLA, SLO, thời gian hoạt động, MTBF, MTTR và ngân sách lỗi.
- Miền lỗi, vùng sẵn sàng, nhân bản đồng bộ và bất đồng bộ; quorum và split-brain.
- Công thức độ sẵn sàng của thành phần nối tiếp và song song; giới hạn của giả định lỗi độc lập.
- Health check, failover, DNS/load balancer và ảnh hưởng của thời gian phát hiện lỗi.

Yêu cầu thực nghiệm

- Triển khai tối thiểu hai bản sao ứng dụng và cụm dữ liệu có cơ chế primary-replica hoặc quorum.
- Xây dựng tối thiểu bốn kịch bản: dừng tiến trình, dừng nút, làm gián đoạn mạng và làm quá tải tài nguyên.
- Lặp lại mỗi kịch bản đủ số lần để tính trung vị và phân vị p95 của thời gian phát hiện, chuyển đổi và khôi phục.
- Đối chiếu kết quả với mô hình lý thuyết; chỉ ra lỗi tương quan, thời gian chờ và phụ thuộc dùng chung làm sai lệch dự báo.

Kết quả và minh chứng cần đạt

- Mô hình cây phụ thuộc và bảng tính độ sẵn sàng kỳ vọng cho ít nhất ba phương án kiến trúc.
- Nhật ký tiêm lỗi, biểu đồ thời gian sự cố và số liệu RTO quan sát được của từng kịch bản.
- Kết luận có căn cứ về cấu hình đạt hoặc chưa đạt SLO; đề xuất thay đổi health check, timeout và số bản sao.
- Bộ kịch bản tự động có thể chạy lại và hướng dẫn tái lập thí nghiệm.

Tài liệu nghiên cứu gợi ý. [6]-[8], [24] và Bài giảng Chương 1.

Công nghệ gợi ý. Kubernetes kind/k3d hoặc Docker Swarm; PostgreSQL streaming replication/Patroni hay Redis Sentinel; Toxiproxy; Chaos Mesh hoặc Litmus; Prometheus và Grafana.

Đề tài 3 Tối ưu tự động co giãn cho dịch vụ web theo tải và theo chỉ số nghiệp vụ

Câu hỏi nghiên cứu. Chính sách co giãn theo CPU có đủ hiệu quả đối với tải biến động nhanh, hay cần kết hợp độ trễ, hàng đợi và số yêu cầu đang chờ?

Mục tiêu của đề tài

- Thiết kế và so sánh ít nhất hai chính sách co giãn cho cùng một dịch vụ.
- Đánh giá quan hệ giữa tốc độ phản ứng, chất lượng dịch vụ và chi phí tài nguyên.
- Xác định ngưỡng và khoảng ổn định giúp giảm dao động tăng - giảm bản sao.

Nội dung lý thuyết cần báo cáo

- Co giãn ngang, co giãn dọc, mở rộng chủ động và phản ứng; cooldown và hysteresis.
- Định luật hàng đợi ở mức nhập môn; quan hệ giữa arrival rate, service time và queue length.
- Các chỉ số p95 latency, throughput, saturation, error rate và cost per request.
- HPA/KEDA hoặc autoscaling của nhà cung cấp; giới hạn do cold start và tài nguyên nền.

Yêu cầu thực nghiệm

- Triển khai dịch vụ stateless có endpoint tạo tải CPU và endpoint mô phỏng công việc theo hàng đợi.
- Chạy cùng một workload dạng bậc thang, spike và chu kỳ với chính sách theo CPU và chính sách theo chỉ số tùy chỉnh.
- Đo số bản sao, thời gian phản ứng, p95 latency, tỷ lệ lỗi, lượng tài nguyên sử dụng và số lần dao động.
- Tối ưu tham số, chạy lại và chứng minh chính sách cuối cùng cải thiện ít nhất một chỉ số mà không làm xấu nghiêm trọng chỉ số còn lại.

Kết quả và minh chứng cần đạt

- Tệp cấu hình hai chính sách co giãn và dashboard thể hiện đồng bộ tải, số bản sao và độ trễ.
- Bộ dữ liệu đo ít nhất ba dạng tải; bảng so sánh trước và sau tối ưu.
- Giải thích nguyên nhân của độ trễ co giãn, quá điều chỉnh hoặc thiếu tài nguyên.
- Khuyến nghị chính sách cho tải dự kiến của một hệ thống cụ thể, kèm giới hạn áp dụng.

Tài liệu nghiên cứu gợi ý. [9], [10], [31] và Bài giảng Chương 1.

Công nghệ gợi ý. Kubernetes HPA, KEDA; Metrics Server; Prometheus Adapter; k6/Locust; Grafana; ứng dụng FastAPI, Node.js hoặc Spring Boot.

Đề tài 4 Thiết kế kết nối hybrid cloud và đánh giá tính di động của một dịch vụ giữa hai môi trường

Câu hỏi nghiên cứu. Một dịch vụ có thể di chuyển giữa hạ tầng tại chỗ và đám mây đến mức nào nếu dữ liệu, danh tính và mạng vẫn tạo ra các phụ thuộc nhà cung cấp?

Mục tiêu của đề tài

- Thiết kế kết nối an toàn giữa hai miền mạng và phân tách rõ luồng quản trị, ứng dụng và dữ liệu.
- Đóng gói một dịch vụ để triển khai được ở hai môi trường từ cùng một kho mã nguồn.

- Đánh giá định lượng chi phí di chuyển, độ trễ, thời gian gián đoạn và các điểm khóa nhà cung cấp.

Nội dung lý thuyết cần báo cáo

- Hybrid cloud, multi-cloud, portability, interoperability và data gravity.
- VPN site-to-site, định tuyến, DNS phân tách, NAT, MTU và kiểm soát truy cập giữa các miền.
- Mẫu active-passive, cloud bursting và workload placement.
- Các lớp phụ thuộc: API, dữ liệu, IAM, giám sát, định dạng cấu hình và dịch vụ được quản lý.

Yêu cầu thực nghiệm

- Tạo hai cụm hoặc hai mạng độc lập, thiết lập kênh VPN và kiểm chứng định tuyến chỉ cho các subnet cần thiết.
- Triển khai một API và cơ sở dữ liệu; thực hiện di chuyển hoặc chuyển hướng lưu lượng giữa hai môi trường.
- Đo RTT, p95 latency, throughput, thời gian đồng bộ dữ liệu và thời gian chuyển đổi.
- Thay thế ít nhất một thành phần phụ thuộc bằng giao diện mở, sau đó đo lại công sức và hành vi vận hành.

Kết quả và minh chứng cần đạt

- Sơ đồ hybrid cloud với bảng định tuyến, vùng tin cậy, cổng dịch vụ và chính sách truy cập.
- Bảng kiểm tính di động theo mã nguồn, dữ liệu, mạng, IAM và quan sát hệ thống.
- Số liệu trước - sau di chuyển; phân tích điểm khóa nhà cung cấp và chi phí thoát dữ liệu giả định.
- Kho cấu hình triển khai cho cả hai môi trường và quy trình chuyển đổi có thể lặp lại.

Tài liệu nghiên cứu gợi ý. [2], [8], [10] và Bài giảng Chương 1.

Công nghệ gợi ý. WireGuard; Kubernetes k3s/kind; Helm hoặc Kustomize; Terraform/OpenTofu; PostgreSQL; MinIO; ExternalDNS tùy chọn.

Đề tài 5 Xây dựng mô hình lựa chọn dịch vụ đám mây dựa trên SLA TCO và rủi ro

Câu hỏi nghiên cứu. Với cùng một bài toán, IaaS, PaaS và serverless tạo ra khác biệt gì về tổng chi phí sở hữu, trách nhiệm vận hành và rủi ro kỹ thuật?

Mục tiêu của đề tài

- Xây dựng bộ tiêu chí quyết định có trọng số từ yêu cầu của một doanh nghiệp giả định.
- Ước tính TCO cho ba phương án triển khai trên cùng đơn vị tải và cùng thời gian phân tích.
- Kiểm chứng một số giả định chi phí và hiệu năng bằng nguyên mẫu nhỏ thay vì chỉ sử dụng bảng giá.

Nội dung lý thuyết cần báo cáo

- CAPEX, OPEX, chi phí trực tiếp - gián tiếp, chi phí nhân lực, dữ liệu ra và khả năng nhàn rỗi.
- SLA, service credit, trách nhiệm chia sẻ và các rủi ro vendor lock-in.

- Mô hình chấm điểm đa tiêu chí, phân tích nhạy cảm và điểm hòa vốn.
- Đặc điểm thanh toán của máy ảo, nền tảng ứng dụng và hàm theo yêu cầu.

Yêu cầu thực nghiệm

- Chọn một API nhỏ và triển khai phiên bản đại diện trên IaaS, PaaS và FaaS hoặc mô phỏng tương đương.
- Tạo ba hồ sơ tải: ổn định, theo giờ hành chính và đột biến; đo độ trễ, cold start và tài nguyên tiêu thụ.
- Lập bảng chi phí tháng theo cùng giả định, gồm compute, storage, request, data transfer, backup, monitoring và công vận hành.
- Thay đổi ít nhất ba tham số đầu vào để xác định điều kiện khiến thứ hạng phương án thay đổi.

Kết quả và minh chứng cần đạt

- Tệp tính TCO có công thức, giả định và nguồn giá; biểu đồ điểm hòa vốn và phân tích nhạy cảm.
- Ma trận SLA - trách nhiệm - rủi ro và bảng chấm điểm đa tiêu chí có giải thích trọng số.
- Kết quả đo nguyên mẫu cùng sai số và giới hạn quy đổi sang quy mô sản xuất.
- Khuyến nghị lựa chọn cho doanh nghiệp giả định, nêu điều kiện cần xem xét lại quyết định.

Tài liệu nghiên cứu gợi ý. [3], [6]-[8], [15] và Bài giảng Chương 1.

Công nghệ gợi ý. AWS/Azure/Google Cloud pricing calculator hoặc Infracost; FastAPI/Node.js; Docker; k6; bảng tính; có thể dùng LocalStack để phát triển trước khi kiểm chứng trên free tier.

Nhóm 2 Mô hình dịch vụ và hạ tầng điện toán đám mây

Phạm vi bám sát Chương 2. Các đề tài yêu cầu sinh viên làm chủ ranh giới trách nhiệm và triển khai thực nghiệm trên IaaS, PaaS, CaaS và FaaS.

Đề tài 6 Xây dựng đám mây IaaS quy mô phòng thí nghiệm bằng nền tảng ảo hóa mã nguồn mở

Câu hỏi nghiên cứu. Các cơ chế template, snapshot, phân bổ tài nguyên và di chuyển máy ảo tạo ra năng lực IaaS như thế nào trên hạ tầng phần cứng giới hạn?

Mục tiêu của đề tài

- Cài đặt và cấu hình nền tảng ảo hóa có khả năng cấp phát máy ảo theo mẫu.
- Đánh giá ảnh hưởng của overcommit CPU và bộ nhớ đến hiệu năng và mức cô lập.
- Xây dựng quy trình cấp phát, sao lưu, khôi phục và thu hồi máy ảo có ghi nhận bằng chứng.

Nội dung lý thuyết cần báo cáo

- Hypervisor loại 1 và loại 2; vCPU, vRAM, virtual disk, bridge và virtual NIC.
- Template, cloud-init, snapshot, clone, migration, thin provisioning và overcommit.
- Đa thuê, cô lập tài nguyên, noisy neighbor và giới hạn của snapshot so với backup.
- Ánh xạ các chức năng của phòng thí nghiệm vào lớp resource abstraction and control của kiến trúc NIST.

Yêu cầu thực nghiệm

- Cài nền tảng trên máy vật lý hoặc nested virtualization; tạo template Linux tích hợp cloud-init.
- Tự động cấp phát ít nhất ba máy ảo thuộc hai mạng logic, áp dụng quota hoặc giới hạn tài nguyên.
- Đo CPU, I/O và độ trễ mạng khi tải đơn và khi xuất hiện noisy neighbor.
- Thực hiện snapshot, backup, restore và nếu hạ tầng cho phép, migration; ghi thời gian và mức gián đoạn.

Kết quả và minh chứng cần đạt

- Cụm IaaS hoặc nút ảo hóa hoạt động, kèm sơ đồ vật lý - logic và danh mục tài nguyên.
- Template chuẩn, kịch bản cấp phát tự động và chính sách đặt tên, quota, vòng đời.
- Bảng benchmark thể hiện mức suy giảm do tranh chấp tài nguyên và giải pháp hạn chế.
- Biên bản kiểm thử khôi phục chứng minh dữ liệu và dịch vụ trở lại đúng trạng thái mong đợi.

Tài liệu nghiên cứu gợi ý. [2], [9], [33] và Bài giảng Chương 2.

Công nghệ gợi ý. Proxmox VE/KVM hoặc OpenStack DevStack; cloud-init; Ansible; fio, iperf3, stress-ng; Prometheus Node Exporter.

Đề tài 7 Triển khai kiến trúc IaaS ba tầng với mạng phân đoạn và cổng truy cập quản trị an toàn

Câu hỏi nghiên cứu. Làm thế nào để công khai dịch vụ web nhưng vẫn giữ tầng ứng dụng và dữ liệu không truy cập trực tiếp từ Internet?

Mục tiêu của đề tài

- Thiết kế VPC/VNet gồm subnet công khai, riêng tư và quản trị với nguyên tắc luồng tối thiểu.
- Triển khai ứng dụng ba tầng có cân bằng tải, NAT hoặc private endpoint và cổng quản trị được kiểm soát.
- Kiểm chứng cả các đường đi được phép lẫn đường đi phải bị chặn.

Nội dung lý thuyết cần báo cáo

- CIDR, subnetting, routing table, Internet gateway, NAT, security group và network ACL.
- Bastion host, session manager, private endpoint và nguyên tắc giảm bề mặt tấn công.
- North-south và east-west traffic; stateful và stateless filtering.
- Shared responsibility trong IaaS và yêu cầu logging cho thay đổi mạng.

Yêu cầu thực nghiệm

- Dựng kiến trúc ba tầng; chỉ load balancer nhận kết nối công khai, máy ứng dụng và cơ sở dữ liệu không có IP công khai.
- Cấu hình luật cho phép theo luồng ứng dụng; chứng minh quét cổng từ vùng ngoài không chạm được tầng dữ liệu.
- Thử một cấu hình sai có kiểm soát, phát hiện bằng công cụ kiểm tra và sửa qua quy trình pull request.
- Đo độ trễ qua từng tầng và kiểm tra hoạt động khi một máy ứng dụng bị dừng.

Kết quả và minh chứng cần đạt

- Sơ đồ mạng, bảng CIDR, bảng luồng và ma trận luật tường lửa có lý do cho từng luật.
- Ứng dụng hoạt động qua endpoint công khai duy nhất; tầng dữ liệu chỉ nhận kết nối từ tầng ứng dụng.
- Bằng chứng kiểm thử positive/negative, kết quả quét trước và sau sửa cấu hình sai.
- Mã IaC hoặc playbook tái tạo toàn bộ hạ tầng.

Tài liệu nghiên cứu gợi ý. [6]-[8], [10], [18] và Bài giảng Chương 2.

Công nghệ gợi ý. AWS VPC/Azure VNet/Google Cloud VPC hoặc mô phỏng bằng Linux network namespaces; Terraform; Nginx; PostgreSQL; nmap; Checkov.

Đề tài 8 Phát triển ứng dụng PaaS có cơ sở dữ liệu được quản lý và đánh giá gánh nặng vận hành

Câu hỏi nghiên cứu. PaaS rút ngắn thời gian triển khai đến mức nào, và đổi lại nhóm phát triển mất quyền kiểm soát hoặc gặp giới hạn gì?

Mục tiêu của đề tài

- Triển khai một ứng dụng web trên PaaS với cấu hình, secret, health check và cơ sở dữ liệu tách biệt.
- Tự động hóa build - deploy từ kho mã nguồn và kiểm chứng rollback.
- So sánh thao tác vận hành với phiên bản IaaS tương đương trên các tiêu chí định lượng.

Nội dung lý thuyết cần báo cáo

- Ranh giới trách nhiệm PaaS; buildpack, runtime, managed database và platform routing.
- Twelve-Factor App, externalized configuration, stateless process và disposable instance.
- Scaling, deployment slot, rolling update, log aggregation và giới hạn runtime.
- Vendor lock-in ở API, dịch vụ dữ liệu, cấu hình và quy trình phát hành.

Yêu cầu thực nghiệm

- Xây dựng ứng dụng CRUD nhỏ có migration cơ sở dữ liệu và kiểm thử tự động.
- Cấu hình pipeline triển khai PaaS; tạo release lỗi để thực hiện rollback và đo thời gian phục hồi.
- Tạo tải, thay đổi số instance và theo dõi p95 latency, error rate, thời gian deploy và công sức cấu hình.
- Triển khai phiên bản IaaS tối thiểu để so sánh số bước, số thành phần phải quản trị và chi phí ước tính.

Kết quả và minh chứng cần đạt

- Ứng dụng PaaS truy cập được, dữ liệu bền vững và secret không nằm trong kho mã nguồn.
- Pipeline CI/CD, lịch sử phát hành, bằng chứng rollback và nhật ký health check.
- Bảng so sánh PaaS - IaaS về thời gian triển khai, quyền kiểm soát, bảo trì, co giãn và chi phí.
- Danh mục phụ thuộc nền tảng cùng phương án giảm khóa nhà cung cấp.

Tài liệu nghiên cứu gợi ý. [3], [6]-[8], [32] và Bài giảng Chương 2.

Công nghệ gợi ý. Azure App Service, Google Cloud Run, AWS Elastic Beanstalk hoặc Render/Fly.io; PostgreSQL managed; GitHub Actions/GitLab CI; k6.

Đề tài 9 Xây dựng hệ thống xử lý ảnh hướng sự kiện bằng serverless và đánh giá cold start

Câu hỏi nghiên cứu. Khi tệp được tải lên theo các nhịp không đều, kiến trúc hướng sự kiện có thể tự động xử lý tin cậy và kiểm soát chi phí ra sao?

Mục tiêu của đề tài

- Thiết kế chuỗi sự kiện từ lưu trữ đối tượng đến hàm xử lý và kho metadata.
- Đảm bảo hàm có tính idempotent, xử lý được retry và đưa lỗi không phục hồi vào dead-letter queue.
- Đo cold start, độ trễ đầu cuối và chi phí theo số lượng sự kiện.

Nội dung lý thuyết cần báo cáo

- FaaS, event source, trigger, stateless execution, timeout, concurrency và at-least-once delivery.
- Idempotency, retry with backoff, dead-letter queue và eventual consistency.
- Cold start, warm invocation, package size và provisioned concurrency.
- Rủi ro bảo mật serverless: quyền quá rộng, injection qua sự kiện, secret và dữ liệu tạm.

Yêu cầu thực nghiệm

- Tải ảnh vào object storage; sự kiện kích hoạt hàm tạo thumbnail, trích metadata và ghi trạng thái.
- Gửi lặp cùng một sự kiện, tạo tệp lỗi và gây timeout để quan sát retry, trùng lặp và dead-letter queue.
- Đo ít nhất 100 lần gọi gồm cold và warm; phân tích p50, p95, tỷ lệ lỗi và thời gian hoàn thành.
- Thay đổi memory/package size hoặc concurrency và đánh giá ảnh hưởng đến hiệu năng - chi phí.

Kết quả và minh chứng cần đạt

- Pipeline serverless hoạt động đầu cuối và không tạo bản ghi trùng khi nhận lại sự kiện.
- Bảng phân phối độ trễ cold/warm, throughput, lỗi và ước tính chi phí cho ba mức tải.
- Chính sách IAM tối thiểu cho từng thành phần và bằng chứng không lưu khóa trong mã.
- Kịch bản kiểm thử lỗi, dashboard và runbook xử lý thông điệp trong hàng đợi lỗi.

Tài liệu nghiên cứu gợi ý. [20], [28], [34] và Bài giảng Chương 2.

Công nghệ gợi ý. AWS Lambda/S3/SQS, Azure Functions/Blob Storage/Queue hoặc Google Cloud Run functions/Cloud Storage/Pub/Sub; LocalStack; Python Pillow; OpenTelemetry.

Đề tài 10 Xây dựng nền tảng CaaS nhỏ cho nhiều nhóm phát triển với quota và cô lập tài nguyên

Câu hỏi nghiên cứu. Một cụm container dùng chung cần những kiểm soát nào để các nhóm tự triển khai mà không ảnh hưởng tài nguyên và bảo mật của nhau?

Mục tiêu của đề tài

- Cung cấp không gian triển khai tách biệt cho ít nhất ba nhóm giả lập.
- Áp dụng quota, limit range, RBAC, network policy và quy tắc nhận image.

- Đánh giá khả năng ngăn chặn tranh chấp tài nguyên và truy cập chéo.

Nội dung lý thuyết cần báo cáo

- CaaS, container runtime, control plane, namespace và multi-tenancy mềm.
- Request, limit, quota, scheduling, eviction và noisy neighbor.
- RBAC, service account, network policy và admission policy.
- Registry, image provenance, secret và trách nhiệm giữa nền tảng với nhóm ứng dụng.

Yêu cầu thực nghiệm

- Tạo ba namespace, ba service account và quyền triển khai khác nhau; giới hạn CPU, RAM và số đối tượng.
- Triển khai ứng dụng mẫu của từng nhóm và chính sách mạng mặc định từ chối, sau đó mở đúng luồng cần thiết.
- Thử vượt quota, dùng image không được phép, đọc secret khác namespace và tạo tải gây tranh chấp.
- Đo mức sử dụng tài nguyên, số yêu cầu bị từ chối và ảnh hưởng đến workload lân cận.

Kết quả và minh chứng cần đạt

- Cụm CaaS có onboarding guide, mẫu namespace và quy trình cấp quyền.
- Bộ manifest/policy chứng minh quota, RBAC, network policy và admission control hoạt động.
- Ma trận kiểm thử được phép - bị từ chối; dashboard tài nguyên theo namespace.
- Phân tích giới hạn của namespace như ranh giới bảo mật và đề xuất khi cần cô lập bằng cụm riêng.

Tài liệu nghiên cứu gợi ý. [9], [10], [19], [23] và Bài giảng Chương 2.

Công nghệ gợi ý. Kubernetes kind/k3d; ResourceQuota, LimitRange, RBAC, NetworkPolicy; Kyverno hoặc OPA Gatekeeper; Harbor; Prometheus/Grafana.

Đề tài 11 So sánh thực nghiệm IaaS PaaS và FaaS cho cùng một dịch vụ API

Câu hỏi nghiên cứu. Khi cùng cung cấp một API, ba mô hình dịch vụ khác nhau như thế nào về hiệu năng, độ co giãn, thời gian phát hành và trách nhiệm vận hành?

Mục tiêu của đề tài

- Triển khai cùng một logic nghiệp vụ trên ba mô hình với đầu vào và dữ liệu tương đương.
- Thiết lập phương pháp đo công bằng, tách thời gian khởi động khỏi thời gian xử lý.
- Đưa ra quy tắc lựa chọn mô hình dựa trên đặc điểm workload.

Nội dung lý thuyết cần báo cáo

- Ranh giới trách nhiệm IaaS, PaaS, FaaS; control versus convenience.
- Cold start, scale-to-zero, instance concurrency và tài nguyên thường trực.
- Chỉ số hiệu năng, khả năng vận hành, portability và tổng chi phí sở hữu.
- Đe dọa tính hợp lệ của benchmark: mạng, caching, cấu hình không tương đương và mẫu tải nhỏ.

Yêu cầu thực nghiệm

- Xây API có endpoint đọc, ghi và tác vụ CPU; dùng cùng bộ dữ liệu và cùng vùng triển khai khi có thể.
- Chạy tải ổn định, burst và idle-then-request; thu thập p50/p95/p99, throughput, lỗi và thời gian scale.
- Ghi số bước triển khai, thời gian sửa lỗi - phát hành, hoạt động vá lỗi và giám sát phải tự quản trị.
- Ước tính chi phí ở ba mức lưu lượng và phân tích điều kiện điểm hòa vốn.

Kết quả và minh chứng cần đạt

- Ba bản triển khai có endpoint tương thích và kho mã nguồn tổ chức để tái lập.
- Bộ dữ liệu benchmark, biểu đồ độ trễ - tải và mô tả sai số.
- Ma trận trách nhiệm, thời gian thao tác và chi phí cho từng mô hình.
- Cây quyết định lựa chọn IaaS/PaaS/FaaS theo tải, kiểm soát, thời gian chạy và năng lực vận hành.

Tài liệu nghiên cứu gợi ý. [1], [3], [20] và Bài giảng Chương 2.

Công nghệ gợi ý. Một VM Linux; Cloud Run/App Service/Elastic Beanstalk; Lambda/Functions; FastAPI/Node.js; Terraform; k6; OpenTelemetry.

Nhóm 3 Lưu trữ cơ sở dữ liệu và xử lý dữ liệu

Phạm vi bám sát Chương 3. Các đề tài đi từ lựa chọn lưu trữ đến cơ sở dữ liệu phân tán, lakehouse, streaming và quản trị metadata.

Đề tài 12 Đánh giá block file và object storage theo mẫu truy cập ứng dụng

Câu hỏi nghiên cứu. Loại lưu trữ nào phù hợp cho cơ sở dữ liệu, thư mục dùng chung và kho tệp lớn khi xét đồng thời độ trễ, thông lượng, ngữ nghĩa truy cập và chi phí?

Mục tiêu của đề tài

- Thiết kế benchmark phản ánh ba mẫu truy cập thay vì chỉ đo tốc độ tuần tự.
- So sánh block, file và object storage trên cùng hạ tầng hoặc cấu hình tài nguyên tương đương.
- Đưa ra tiêu chí lựa chọn gắn với workload và vòng đời dữ liệu.

Nội dung lý thuyết cần báo cáo

- Block, file, object; POSIX semantics, object key, metadata và consistency.
- IOPS, throughput, latency, queue depth, kích thước khối và small-file problem.
- Độ bền, khả dụng, replication, erasure coding và storage class.
- Lifecycle, versioning, retention, egress và chi phí request.

Yêu cầu thực nghiệm

- Dựng block volume, NFS và S3-compatible object storage; xác định tài nguyên nền và giới hạn benchmark.
- Chạy workload tuần tự lớn, ngẫu nhiên 4 KiB, nhiều tệp nhỏ và upload/download đối tượng song song.

- Thử lỗi một nút hoặc ngắt kết nối; kiểm tra checksum và khả năng truy cập sau phục hồi.
- Tính chi phí giả định cho 1 TB theo tần suất truy cập nóng, mát và lưu trữ lâu dài.

Kết quả và minh chứng cần đạt

- Kịch bản benchmark có warm-up, số lần lặp và mô tả môi trường.
- Biểu đồ IOPS, MB/s, p95 latency, CPU, dung lượng hữu dụng và tỷ lệ lỗi.
- Bảng ánh xạ workload - loại lưu trữ - lý do - rủi ro.
- Chính sách vòng đời object mẫu và kiểm chứng versioning/checksum.

Tài liệu nghiên cứu gợi ý. [35]-[37] và Bài giảng Chương 3.

Công nghệ gợi ý. Linux LVM; NFS; MinIO; fio; s3-benchmark hoặc Warp; iperf3; Prometheus/Grafana.

Đề tài 13 Thiết kế cơ sở dữ liệu quan hệ sẵn sàng cao với sao lưu phục hồi đến thời điểm

Câu hỏi nghiên cứu. Cơ chế replication và backup cần phối hợp ra sao để vừa giảm gián đoạn vừa tránh mất dữ liệu do lỗi thao tác?

Mục tiêu của đề tài

- Triển khai cơ sở dữ liệu primary-replica có giám sát độ trễ sao chép.
- Thiết lập backup toàn phần, lưu WAL và phục hồi đến thời điểm xác định.
- Phân biệt vai trò của high availability với backup qua thực nghiệm lỗi.

Nội dung lý thuyết cần báo cáo

- ACID, transaction log, replication đồng bộ/bất đồng bộ, read replica và failover.
- RPO, RTO, backup full/incremental, PITR, retention và kiểm tra khả năng phục hồi.
- Lỗi phần cứng, lỗi logic, xóa nhầm và ransomware; vì sao replica không thay thế backup.
- Connection pooling, consistency khi đọc replica và split-brain.

Yêu cầu thực nghiệm

- Dựng primary và ít nhất một replica; tạo tải ghi - đọc liên tục và đo replication lag.
- Dừng primary, chuyển đổi sang replica và đo RTO cùng số giao dịch có nguy cơ mất.
- Tạo bảng dữ liệu, xóa nhầm tại mốc đã ghi nhận, phục hồi đến thời điểm ngay trước thao tác.
- Kiểm tra checksum/số bản ghi và chạy kiểm thử ứng dụng sau phục hồi.

Kết quả và minh chứng cần đạt

- Kiến trúc HA và sơ đồ luồng backup/WAL; runbook failover và PITR.
- Bảng RPO/RTO mục tiêu - quan sát, replication lag và thời gian phục hồi.
- Bằng chứng dữ liệu được phục hồi đúng điểm, cùng nhật ký kiểm tra tính toàn vẹn.
- Phân tích rủi ro còn lại và đề xuất lịch backup, retention, mã hóa và bản sao ngoài miền lỗi.

Tài liệu nghiên cứu gợi ý. [38], [24] và Bài giảng Chương 3, Chương 4.

Công nghệ gợi ý. PostgreSQL; Patroni/repmgr; pgBackRest hoặc WAL-G; HAProxy; MinIO; Prometheus postgres_exporter.

Đề tài 14 Thiết kế mô hình dữ liệu NoSQL và kiểm chứng đánh đổi CAP cho hệ thống danh mục sản phẩm

Câu hỏi nghiên cứu. Partition key, replication và consistency level ảnh hưởng thế nào đến khả năng mở rộng, tính đúng đắn và độ trễ của truy vấn?

Mục tiêu của đề tài

- Thiết kế mô hình dữ liệu theo truy vấn của ứng dụng thay vì chuyển nguyên lược đồ quan hệ sang NoSQL.
- Phát hiện và khắc phục hot partition bằng dữ liệu đo.
- Minh họa hành vi nhất quán khi xảy ra phân vùng mạng hoặc replica chậm.

Nội dung lý thuyết cần báo cáo

- Key-value, document, wide-column và graph; denormalization và query-driven modeling.
- CAP trong điều kiện có network partition; consistency level và quorum.
- Partition key, sharding, replication factor, hot key và secondary index.
- Eventual consistency, read-your-writes, conflict và idempotent update.

Yêu cầu thực nghiệm

- Xây bộ dữ liệu sản phẩm, tồn kho và sự kiện xem; thiết kế ít nhất hai phương án partition key.
- Nạp dữ liệu và tạo tải đọc/ghi có phân bố lệch; đo throughput, p95 latency và tải từng nút.
- Gây gián đoạn mạng hoặc làm replica chậm; thử các consistency level và ghi nhận stale read/lỗi.
- Điều chỉnh lược đồ hoặc sharding, chạy lại cùng workload và so sánh.

Kết quả và minh chứng cần đạt

- Tài liệu access pattern, lược đồ NoSQL và lý do chọn partition key.
- Biểu đồ phân bố tải, độ trễ, throughput và tỷ lệ stale/error theo cấu hình.
- Bằng chứng hot partition trước - sau tối ưu.
- Kết luận đúng phạm vi về CAP, tránh gắn nhãn CP/AP khi không nêu điều kiện vận hành.

Tài liệu nghiên cứu gợi ý. [39], [40] và Bài giảng Chương 3.

Công nghệ gợi ý. MongoDB replica set hoặc Apache Cassandra/ScyllaDB; YCSB; Toxiproxy; Python; Grafana.

Đề tài 15 Xây dựng data lakehouse cho dữ liệu bán hàng bằng quy trình ETL và ELT

Câu hỏi nghiên cứu. Kiến trúc lakehouse cải thiện khả năng lưu trữ dữ liệu thô, quản trị bảng và phân tích so với thư mục data lake không có lớp giao dịch như thế nào?

Mục tiêu của đề tài

- Xây pipeline ingest dữ liệu đa định dạng vào các lớp bronze, silver và gold.
- Áp dụng định dạng bảng hỗ trợ schema evolution, partitioning và time travel.
- So sánh ETL và ELT trên cùng một yêu cầu làm sạch và tổng hợp.

Nội dung lý thuyết cần báo cáo

- Data lake, warehouse, lakehouse; schema-on-read và schema-on-write.
- ETL, ELT, phân lớp dữ liệu, partition pruning và small-file problem.
- ACID table format, metadata, snapshot, schema evolution và compaction.
- Chất lượng dữ liệu, lineage, idempotent pipeline và quyền truy cập theo vùng dữ liệu.

Yêu cầu thực nghiệm

- Tạo dữ liệu đơn hàng CSV, sự kiện JSON và thông tin sản phẩm; nạp vào object storage.
- Xây pipeline làm sạch, loại trùng, chuẩn hóa schema và tạo bảng tổng hợp doanh thu.
- Thử thêm cột, cập nhật bản ghi, truy vấn phiên bản cũ và compact tệp nhỏ.
- Đo thời gian ingest, transform, query, dung lượng và số tệp trước - sau tối ưu.

Kết quả và minh chứng cần đạt

- Data lakehouse chạy được với cấu trúc lớp dữ liệu và catalog rõ ràng.
- Notebook/job tái lập pipeline; kiểm thử chất lượng dữ liệu và xử lý chạy lại không tạo trùng.
- Biểu đồ hiệu năng query, dung lượng, số tệp và tác động của partition/compaction.
- Báo cáo so sánh ETL - ELT và giải thích khi nào cần warehouse hoặc lakehouse.

Tài liệu nghiên cứu gợi ý. [25], [41], [42] và Bài giảng Chương 3.

Công nghệ gợi ý. MinIO; Apache Spark; Delta Lake hoặc Apache Iceberg; Nessie/Hive Metastore; DuckDB/Trino; dbt tùy chọn.

Đề tài 16 Xây dựng pipeline xử lý dữ liệu thời gian thực và đánh giá độ trễ đầu cuối

Câu hỏi nghiên cứu. Window, watermark, checkpoint và cơ chế giao nhận thông điệp ảnh hưởng thế nào đến độ trễ và độ chính xác khi dữ liệu đến muộn?

Mục tiêu của đề tài

- Xây pipeline ingest - xử lý - lưu trữ cho luồng sự kiện liên tục.
- Xử lý dữ liệu đến trễ và khôi phục trạng thái sau khi tiến trình xử lý bị dừng.
- Đo throughput, consumer lag, end-to-end latency và độ chính xác của kết quả cửa sổ.

Nội dung lý thuyết cần báo cáo

- Batch, stream, Lambda/Kappa architecture; event time và processing time.
- Partition, offset, consumer group, at-most/at-least/exactly-once semantics.
- Window, watermark, state, checkpoint và backpressure.
- Quan sát pipeline qua lag, throughput, lỗi và thời gian checkpoint.

Yêu cầu thực nghiệm

- Sinh luồng sự kiện cảm biến hoặc giao dịch có timestamp, khóa và một tỷ lệ sự kiện đến muộn.
- Tính thống kê theo cửa sổ, ghi kết quả vào kho truy vấn và hiển thị dashboard gần thời gian thực.
- Tăng dần tốc độ phát, theo dõi lag/backpressure; dừng và khởi động lại processor để kiểm tra checkpoint.

- Thay đổi watermark và số partition, đo tác động đến độ trễ, tài nguyên và số bản ghi muộn bị loại.

Kết quả và minh chứng cần đạt

- Pipeline hoạt động đầu cuối với generator có thể kiểm soát tốc độ và tỷ lệ dữ liệu đến muộn.
- Dashboard throughput, lag, p95 end-to-end latency và trạng thái checkpoint.
- Bảng so sánh cấu hình partition/watermark; kiểm chứng kết quả bằng tập dữ liệu chuẩn.
- Phân tích semantics đạt được trong cấu hình thực tế, không chỉ theo mô tả lý thuyết.

Tài liệu nghiên cứu gợi ý. [25], [43], [44] và Bài giảng Chương 3.

Công nghệ gợi ý. Apache Kafka hoặc Redpanda; Spark Structured Streaming hoặc Apache Flink; PostgreSQL/ClickHouse; Prometheus/Grafana.

Đề tài 17 Xây dựng danh mục dữ liệu và truy vết dòng dữ liệu cho môi trường phân tích đám mây

Câu hỏi nghiên cứu. Metadata, ownership, classification và lineage giúp ngăn data lake trở thành data swamp như thế nào?

Mục tiêu của đề tài

- Thu thập metadata tự động từ nhiều nguồn và xây danh mục dữ liệu có thể tìm kiếm.
- Thiết lập chủ sở hữu, mô tả, thẻ dữ liệu nhạy cảm và quy tắc chất lượng cho tài sản dữ liệu.
- Hiển thị lineage từ nguồn đến bảng tổng hợp và đánh giá tác động khi thay đổi schema.

Nội dung lý thuyết cần báo cáo

- Technical, business và operational metadata; data catalog, glossary và ownership.
- Data lineage, impact analysis, data quality, profiling và schema evolution.
- Phân loại PII, quyền truy cập theo vai trò và nguyên tắc tối thiểu.
- Data governance tập trung và data mesh; dữ liệu như một sản phẩm.

Yêu cầu thực nghiệm

- Kết nối catalog với ít nhất hai nguồn như PostgreSQL và object storage hoặc dashboard BI.
- Ingest metadata, bổ sung glossary, owner, domain, tag PII và mô tả cho tập dữ liệu trọng tâm.
- Tạo pipeline biến đổi có lineage; thay đổi một cột nguồn để thực hiện impact analysis.
- Tạo quy tắc chất lượng và cảnh báo cho null, uniqueness hoặc freshness; ghi nhận sự cố giả lập.

Kết quả và minh chứng cần đạt

- Catalog tra cứu được với tối thiểu 15 tài sản dữ liệu có owner, mô tả và phân loại.
- Sơ đồ lineage đầu cuối và biên bản impact analysis cho một thay đổi schema.
- Dashboard kiểm thử chất lượng, bằng chứng phát hiện dữ liệu không đạt và quy trình xử lý.
- Ma trận quyền/role và hướng dẫn duy trì metadata sau khi đồ án kết thúc.

Tài liệu nghiên cứu gợi ý. [45] và Bài giảng Chương 3, Chương 4.

Công nghệ gợi ý. OpenMetadata hoặc DataHub; PostgreSQL; MinIO; dbt; Airflow; Great Expectations hoặc Soda Core.

Nhóm 4 An toàn bảo mật và quản trị đám mây

Phạm vi bám sát Chương 4. Các đề tài dùng nguyên tắc danh tính, Zero Trust, mã hóa, DevSecOps, giám sát và phục hồi để bảo vệ workload.

Đề tài 18 Thiết kế IAM theo nguyên tắc quyền tối thiểu và kiểm chứng bằng nhật ký kiểm toán

Câu hỏi nghiên cứu. Có thể thay khóa truy cập dài hạn và quyền quản trị rộng bằng vai trò, thông tin xác thực tạm thời và quy trình phê duyệt như thế nào?

Mục tiêu của đề tài

- Xây mô hình người dùng, nhóm, vai trò và tài nguyên cho một doanh nghiệp giả định.
- Áp dụng MFA, quyền tối thiểu, tách nhiệm vụ và thông tin xác thực ngắn hạn.
- Kiểm chứng quyết định cho phép/từ chối và truy vết hành động qua audit log.

Nội dung lý thuyết cần báo cáo

- Authentication, authorization, accounting; RBAC, ABAC và policy evaluation.
- Human identity, workload identity, federation, MFA và temporary credential.
- Least privilege, separation of duties, privilege escalation và policy boundary.
- Access review, key rotation, audit trail và shared responsibility.

Yêu cầu thực nghiệm

- Mô hình hóa ít nhất bốn vai trò: quản trị, phát triển, kiểm toán và workload; định nghĩa hành động cần thiết.
- Cấu hình đăng nhập liên kết/MFA nếu nền tảng hỗ trợ; workload dùng role thay cho khóa tĩnh.
- Thực hiện bộ ca kiểm thử positive/negative, gồm đọc dữ liệu, thay đổi mạng, xem log và cố nâng quyền.
- Phân tích audit log để xác định chủ thể, hành động, tài nguyên, thời gian và kết quả.

Kết quả và minh chứng cần đạt

- Ma trận chủ thể - tài nguyên - hành động và bộ policy có quản lý phiên bản.
- Tối thiểu 12 ca kiểm thử quyền với kết quả mong đợi và thực tế.
- Bằng chứng workload không chứa khóa dài hạn; báo cáo truy vết một phiên thao tác.
- Danh sách quyền dư thừa phát hiện được và phiên bản policy đã tinh chỉnh.

Tài liệu nghiên cứu gợi ý. [18], [22], [46] và Bài giảng Chương 4.

Công nghệ gợi ý. IAM của AWS/Azure/Google Cloud hoặc Keycloak + Open Policy Agent; CloudTrail/Activity Log/Audit Logs; policy simulator; Prowler tùy chọn.

Đề tài 19 Xây dựng mô hình Zero Trust cho truy cập ứng dụng cloud native

Câu hỏi nghiên cứu. Khi vị trí mạng không còn được xem là bằng chứng tin cậy, hệ thống phải xác minh danh tính người dùng và dịch vụ ở những điểm nào?

Mục tiêu của đề tài

- Thiết kế luồng truy cập dựa trên danh tính, trạng thái phiên và chính sách theo tài nguyên.
- Bảo vệ giao tiếp người dùng - ứng dụng và service-to-service mà không dựa vào mạng nội bộ mặc định tin cậy.
- Chứng minh truy cập bị thu hồi hoặc thu hẹp khi danh tính, vai trò hoặc ngữ cảnh thay đổi.

Nội dung lý thuyết cần báo cáo

- Nguyên tắc Zero Trust, policy decision point, policy enforcement point và continuous evaluation.
- OIDC/OAuth 2.0, token, claim, workload identity, mTLS và service identity.
- Microsegmentation, default deny và network policy so với identity-aware policy.
- Zero Trust cho cloud-native và multi-cloud theo NIST SP 800-207A.

Yêu cầu thực nghiệm

- Triển khai identity provider, reverse proxy/policy enforcement và hai dịch vụ có mức quyền khác nhau.
- Yêu cầu xác thực cho người dùng; cấp quyền theo role/claim và thời hạn token.
- Áp dụng mTLS hoặc service identity cho giao tiếp nội bộ; network policy mặc định từ chối.
- Thử token hết hạn, người dùng bị thu hồi vai trò, dịch vụ giả mạo và truy cập từ luồng không được phép.

Kết quả và minh chứng cần đạt

- Sơ đồ thành phần Zero Trust và sequence diagram cho quyết định truy cập.
- Bộ policy danh tính và mạng; ít nhất 10 ca kiểm thử cho phép/từ chối.
- Audit log liên kết được người dùng, client, dịch vụ và tài nguyên.
- Phân tích phần nào mới chỉ là nguyên mẫu và yêu cầu bổ sung để đạt continuous verification trong sản xuất.

Tài liệu nghiên cứu gợi ý. [16], [17] và Bài giảng Chương 4.

Công nghệ gợi ý. Keycloak; oauth2-proxy; Envoy/Istio hoặc Linkerd; SPIFFE/SPIRE tùy chọn; Kubernetes NetworkPolicy; OPA.

Đề tài 20 Bảo vệ dữ liệu bằng mã hóa quản lý khóa và vòng đời bí mật

Câu hỏi nghiên cứu. Mã hóa chỉ có ý nghĩa bảo vệ đến đâu nếu khóa, secret ứng dụng và quyền giải mã không được quản lý tách biệt?

Mục tiêu của đề tài

- Áp dụng mã hóa khi truyền, khi lưu và mã hóa cấp ứng dụng cho một trường dữ liệu nhạy cảm.
- Thiết kế vòng đời secret gồm tạo, phân phối, xoay vòng, thu hồi và kiểm toán.
- Kiểm chứng ứng dụng tiếp tục hoạt động trong quá trình xoay vòng khóa/secret có kiểm soát.

Nội dung lý thuyết cần báo cáo

- TLS, encryption at rest, envelope encryption, data key và key-encryption key.

- KMS/HSM, rotation, versioning, separation of duties và crypto-shredding.
- Secret manager, dynamic secret, lease, revocation và nguy cơ secret trong image/log/Git.
- Phân loại dữ liệu, quyền decrypt và audit trail.

Yêu cầu thực nghiệm

- Triển khai API lưu dữ liệu; bật TLS, mã hóa volume/object và mã hóa trường nhạy cảm bằng envelope encryption.
- Đưa thông tin kết nối vào secret manager; ứng dụng lấy secret bằng danh tính workload.
- Xoay vòng mật khẩu cơ sở dữ liệu hoặc data key, chứng minh phiên bản mới hoạt động và phiên bản cũ bị thu hồi theo kế hoạch.
- Quét kho mã nguồn, image và log để phát hiện secret; tạo cảnh báo cho hành động giải mã bất thường.

Kết quả và minh chứng cần đạt

- Sơ đồ luồng dữ liệu và ranh giới nơi dữ liệu ở dạng rõ/mã hóa.
- Policy KMS/secret tối thiểu, log sử dụng khóa và biên bản rotation.
- Bằng chứng secret không nằm trong mã nguồn, image hoặc tệp cấu hình công khai.
- Kế hoạch xử lý mất khóa/khóa bị lộ và đánh giá tác động đến khả năng phục hồi.

Tài liệu nghiên cứu gợi ý. [18], [47], [48] và Bài giảng Chương 4.

Công nghệ gợi ý. HashiCorp Vault hoặc cloud KMS/Secrets Manager; PostgreSQL; Docker/Kubernetes; Gitleaks; TruffleHog; OpenSSL.

Đề tài 21 Xây dựng đường ống DevSecOps phát hiện cấu hình sai và lỗ hổng trước triển khai

Câu hỏi nghiên cứu. Những kiểm soát tự động nào nên chặn bản phát hành, và ngưỡng nào chỉ nên cảnh báo để không làm tê liệt quy trình phát triển?

Mục tiêu của đề tài

- Tích hợp kiểm tra mã nguồn, dependency, container image, secret và IaC vào CI.
- Định nghĩa security gate theo mức độ nghiêm trọng và ngoại lệ có thời hạn.
- Đo tỷ lệ phát hiện, thời gian pipeline và chất lượng khắc phục qua một bộ lỗi có chủ đích.

Nội dung lý thuyết cần báo cáo

- Shift-left, defense in depth và phạm vi SAST, SCA, secret scanning, IaC scanning, image scanning.
- CVSS, exploitability, false positive, risk acceptance và policy as code.
- SBOM, provenance, image signing và chuỗi cung ứng phần mềm.
- Cấu hình cloud sai, benchmark và kiểm soát theo CSA CCM/CIS.

Yêu cầu thực nghiệm

- Tạo ứng dụng và IaC mẫu chứa tập lỗi kiểm soát: dependency cũ, secret giả, Dockerfile yếu, security group rộng và quyền quá mức.
- Xây pipeline chạy tối thiểu bốn lớp kiểm tra; xuất báo cáo máy đọc được và bản tóm tắt cho pull request.
- Thiết lập gate; xác minh commit lỗi bị chặn, sau đó sửa và chứng minh pipeline vượt qua.

- Ghi thời gian chạy, số lỗi thật/giả và ảnh hưởng của cache hoặc chạy song song.

Kết quả và minh chứng cần đạt

- Pipeline DevSecOps hoàn chỉnh với policy chặn rõ ràng và cơ chế ngoại lệ có chủ sở hữu, thời hạn.
- Bảng lỗi gieo cài - công cụ phát hiện - kết quả - cách sửa.
- SBOM và báo cáo lỗ hổng của image trước - sau khắc phục.
- Phân tích thời gian pipeline và đề xuất cân bằng tốc độ phát hành với độ bao phủ.

Tài liệu nghiên cứu gợi ý. [18], [19], [23], [49], [50] và Bài giảng Chương 4, Chương 5.

Công nghệ gợi ý. GitHub Actions/GitLab CI; Semgrep; Trivy; Grype/Syft; Gitleaks; Checkov/tfsec; Cosign; OPA/Conftest.

Đề tài 22 Xây dựng giám sát tập trung và diễn tập ứng phó sự cố trên đám mây

Câu hỏi nghiên cứu. Từ hàng nghìn log và sự kiện, nhóm vận hành cần tương quan tín hiệu nào để phát hiện và điều tra một hành vi bất thường có căn cứ?

Mục tiêu của đề tài

- Thu thập log ứng dụng, hệ điều hành, danh tính và mạng vào một hệ thống tập trung.
- Xây quy tắc phát hiện cho một kịch bản tấn công giả lập và giảm cảnh báo nhiễu.
- Thực hiện đầy đủ chu trình xác minh, khoanh vùng, loại bỏ và phục hồi.

Nội dung lý thuyết cần báo cáo

- Logging, monitoring, detection engineering, SIEM và yêu cầu đồng bộ thời gian.
- Chu trình ứng phó sự cố; evidence, timeline, chain of custody ở mức phòng thí nghiệm.
- Use case phát hiện, baseline, threshold, correlation và false positive.
- Cloud audit log, network flow log, application log và giới hạn quan sát.

Yêu cầu thực nghiệm

- Thu thập ít nhất bốn nguồn log và chuẩn hóa trường thời gian, danh tính, địa chỉ, hành động, tài nguyên.
- Mô phỏng hợp pháp trong lab: đăng nhập thất bại dồn dập, truy cập bucket trái phép hoặc credential bị lạm dụng.
- Xây quy tắc và dashboard; chạy traffic bình thường để đo false positive, sau đó chạy kịch bản sự cố.
- Thực thi playbook, bảo toàn log, lập timeline và kiểm chứng dịch vụ sau phục hồi.

Kết quả và minh chứng cần đạt

- Kiến trúc thu thập log, chính sách lưu giữ và đồng bộ thời gian.
- Dashboard và cảnh báo có bằng chứng phát hiện sự cố; chỉ số MTTD/MTTA/MTTR trong diễn tập.
- Timeline sự cố, danh sách chỉ dấu, quyết định khoanh vùng và bài học sau sự cố.
- Quy tắc phát hiện ở dạng mã hoặc cấu hình có thể tái sử dụng.

Tài liệu nghiên cứu gợi ý. [21], [51] và Bài giảng Chương 4.

Công nghệ gợi ý. Wazuh hoặc OpenSearch/Elastic Stack; Falco; cloud audit logs; Zeek/Suricata tùy chọn; Sigma rules.

Đề tài 23 Thiết kế và kiểm chứng kế hoạch sao lưu phục hồi thảm họa cho ứng dụng cloud native

Câu hỏi nghiên cứu. Kế hoạch phục hồi có thực sự đạt RPO/RTO nếu chỉ tồn tại trên tài liệu mà chưa thử khôi phục ứng dụng, cấu hình và dữ liệu?

Mục tiêu của đề tài

- Phân tích tác động nghiệp vụ và đặt RPO/RTO cho từng thành phần.
- Triển khai sao lưu tách miền lỗi cho dữ liệu và cấu hình triển khai.
- Diễn tập mất cụm hoặc mất vùng và đo thời gian phục hồi đầu cuối.

Nội dung lý thuyết cần báo cáo

- Business impact analysis, RPO, RTO, retention và backup 3-2-1.
- Pilot light, warm standby, active-passive, active-active và chi phí sẵn sàng.
- Backup ứng dụng nhất quán, database-consistent snapshot, immutability và mã hóa.
- Runbook, quyền kích hoạt thảm họa, truyền thông và kiểm thử định kỳ.

Yêu cầu thực nghiệm

- Đặt mục tiêu RPO/RTO; sao lưu cơ sở dữ liệu, object và tài nguyên Kubernetes/IaC sang vị trí độc lập.
- Xóa có kiểm soát namespace/cụm hoặc giả lập mất vùng; kích hoạt runbook trên môi trường dự phòng.
- Đo thời gian phát hiện, quyết định, cấp phát, phục hồi dữ liệu, kiểm thử và chuyển lưu lượng.
- So sánh hash/số bản ghi/chức năng ứng dụng; ghi nhận dữ liệu nằm ngoài RPO nếu có.

Kết quả và minh chứng cần đạt

- BIA ngắn, bảng RPO/RTO và chiến lược DR có lý do lựa chọn.
- Backup được mã hóa, có chính sách retention và bằng chứng khôi phục thành công.
- Báo cáo diễn tập nêu thời gian từng bước, sai lệch so với mục tiêu và điểm nghẽn.
- Runbook đã chỉnh sửa sau diễn tập, danh sách kiểm thử tính toàn vẹn và kế hoạch kiểm thử lại.

Tài liệu nghiên cứu gợi ý. [24], [52] và Bài giảng Chương 4.

Công nghệ gợi ý. Velero; Restic/Kopia; pgBackRest; MinIO object lock; Terraform; Kubernetes; DNS/load balancer mô phỏng.

Nhóm 5 Phát triển triển khai và vận hành ứng dụng

Phạm vi bám sát Chương 5. Các đề tài tích hợp container, Kubernetes, CI/CD, IaC, GitOps, observability, FinOps và dịch vụ AI trên cloud.

Đề tài 24 Hiện đại hóa ứng dụng nguyên khối thành kiến trúc cloud native có đo kiểm

Câu hỏi nghiên cứu. Tách dịch vụ ở mức nào tạo lợi ích triển khai và co giãn thực sự mà không làm độ phức tạp phân tán vượt quá năng lực nhóm?

Mục tiêu của đề tài

- Phân tích miền nghiệp vụ và chọn một ranh giới dịch vụ có căn cứ.
- Hiện đại hóa theo từng bước, giữ hệ thống hoạt động trong quá trình chuyển đổi.
- So sánh khả năng phát hành, mở rộng và phục hồi trước - sau thay đổi.

Nội dung lý thuyết cần báo cáo

- Monolith, modular monolith, microservices; coupling, cohesion và bounded context.
- Strangler pattern, API gateway, asynchronous messaging và database per service.
- Stateless service, externalized configuration, resilience pattern và distributed transaction.
- Các chiến lược 7R và tiêu chí quyết định rehost, replatform, refactor.

Yêu cầu thực nghiệm

- Chọn ứng dụng nguyên khối nhỏ; lập dependency map và đo baseline build/deploy, latency, lỗi.
- Tách một chức năng phù hợp thành service/API hoặc worker bất đồng bộ; đóng gói container.
- Triển khai cả hai phiên bản; thử phát hành độc lập, scale riêng và dừng service để quan sát phạm vi lỗi.
- Đo thay đổi về kích thước release, thời gian deploy, p95 latency, tài nguyên và số điểm vận hành.

Kết quả và minh chứng cần đạt

- Tài liệu quyết định kiến trúc và ranh giới dịch vụ; sơ đồ trước - sau.
- Hai phiên bản chạy được với bộ kiểm thử hồi quy chung.
- Bảng đo lợi ích và chi phí phức tạp; phân tích phần nào nên giữ nguyên khối.
- Lộ trình hiện đại hóa tiếp theo có ưu tiên, phụ thuộc và rủi ro.

Tài liệu nghiên cứu gợi ý. [11], [32], [53] và Bài giảng Chương 5.

Công nghệ gợi ý. FastAPI/Spring Boot/Node.js; Docker Compose/Kubernetes; RabbitMQ/Kafka; PostgreSQL; OpenTelemetry; k6.

Đề tài 25 Bảo vệ chuỗi cung ứng container bằng SBOM ký image và xác minh chính sách

Câu hỏi nghiên cứu. Làm sao chứng minh image được triển khai đúng là sản phẩm của pipeline tin cậy và không chứa lỗ hổng vượt ngưỡng?

Mục tiêu của đề tài

- Tạo image tối giản, tái lập được và không chạy bằng quyền root.
- Sinh SBOM, quét lỗ hổng, ký image và xác minh chữ ký trước triển khai.
- Chặn image không ký, dùng tag không cố định hoặc có lỗ hổng nghiêm trọng.

Nội dung lý thuyết cần báo cáo

- Image layer, base image, multi-stage build, digest và registry.
- SBOM, CVE, vulnerability database, provenance và SLSA.
- Ký artifact, keyless signing, transparency log và admission control.
- Rủi ro dependency confusion, typosquatting, secret trong layer và image drift.

Yêu cầu thực nghiệm

- Xây image cơ sở có chủ đích chưa tối ưu; quét kích thước, package, CVE và cấu hình.
- Dùng multi-stage, pin digest, non-root và loại dependency thừa; tạo lại SBOM và đo cải thiện.
- Ký image từ CI; cấu hình cluster chỉ nhận image có chữ ký và digest phù hợp.
- Thử triển khai image không ký, image bị thay tag và image có CVE vượt ngưỡng.

Kết quả và minh chứng cần đạt

- Dockerfile trước - sau, số liệu kích thước, package và lỗ hổng.
- SBOM theo chuẩn SPDX hoặc CycloneDX; chữ ký và bằng chứng xác minh.
- Admission policy cùng ma trận ca kiểm thử cho phép/từ chối.
- Quy trình xử lý CVE có SLA, ngoại lệ và cơ chế cập nhật base image.

Tài liệu nghiên cứu gợi ý. [11], [23], [49], [50], [54] và Bài giảng Chương 5.

Công nghệ gợi ý. Docker BuildKit; Syft; Grype/Trivy; Cosign; GitHub Actions/GitLab CI; Kyverno hoặc Gatekeeper; Harbor/GHCR.

Đề tài 26 Triển khai ứng dụng trên Kubernetes với tự phục hồi co giãn và cập nhật không gián đoạn

Câu hỏi nghiên cứu. Các controller và health probe của Kubernetes phối hợp như thế nào để duy trì trạng thái mong muốn khi pod lỗi, tải tăng hoặc phiên bản mới gặp sự cố?

Mục tiêu của đề tài

- Triển khai ứng dụng theo cấu hình khai báo, tách config/secret và sử dụng storage phù hợp.
- Cấu hình self-healing, rolling update, rollback và HPA.
- Đo tính sẵn sàng và độ trễ trong các tình huống lỗi và cập nhật.

Nội dung lý thuyết cần báo cáo

- Pod, Deployment, Service, ConfigMap, Secret, Ingress và PersistentVolume.
- Reconciliation loop, desired state, liveness/readiness/startup probe.
- RollingUpdate, maxSurge, maxUnavailable, rollback và PodDisruptionBudget.
- Requests/limits, HPA và quan hệ giữa scheduler, autoscaler với metrics.

Yêu cầu thực nghiệm

- Triển khai ứng dụng ít nhất hai replica, readiness/liveness probe, resource request/limit và ingress.

- Xóa pod, gây lỗi health endpoint và drain node; theo dõi self-healing và mức gián đoạn.
- Phát hành phiên bản mới, tạo lỗi, rollback; tạo tải để HPA tăng rồi giảm replica.
- Đo availability, p95 latency, số lỗi, thời gian rollout/rollback và tài nguyên.

Kết quả và minh chứng cần đạt

- Bộ manifest/Helm chart có cấu trúc, không chứa secret rõ.
- Dashboard thể hiện replica, restart, CPU/memory, độ trễ và lỗi theo thời gian.
- Bảng kiểm thử pod/node/rollout/autoscaling với tiêu chí đạt.
- Phân tích cấu hình probe và resource sai có thể làm giảm độ tin cậy như thế nào.

Tài liệu nghiên cứu gợi ý. [9], [10], [19] và Bài giảng Chương 5.

Công nghệ gợi ý. Kubernetes kind/k3d/minikube; Helm; Nginx Ingress; Metrics Server; Prometheus/Grafana; k6; Chaos Mesh tùy chọn.

Đề tài 27 Xây dựng CI CD với chiến lược blue green và canary có tự động rollback

Câu hỏi nghiên cứu. Có thể giảm blast radius của bản phát hành bằng cách kiểm soát lưu lượng và dùng chỉ số dịch vụ để ra quyết định tự động ra sao?

Mục tiêu của đề tài

- Tự động hóa kiểm thử, build, quét, phát hành và triển khai từ một commit.
- Thực hiện blue-green và canary cho cùng ứng dụng.
- Dùng metric để tiếp tục hoặc rollback thay vì quyết định thủ công hoàn toàn.

Nội dung lý thuyết cần báo cáo

- Continuous integration, delivery, deployment; artifact immutability và environment promotion.
- Rolling, recreate, blue-green, canary; blast radius và backward compatibility.
- Feature flag, database migration an toàn và contract testing.
- Deployment metric, error budget, automated analysis và rollback.

Yêu cầu thực nghiệm

- Xây pipeline từ kiểm thử đến image có version; triển khai vào môi trường staging rồi production giả lập.
- Thực hiện blue-green, đo thời gian chuyển đổi và khả năng quay lại phiên bản cũ.
- Thực hiện canary 5%-25%-50%-100%; phát hành một phiên bản làm tăng lỗi có kiểm soát.
- Cấu hình phân tích tỷ lệ lỗi/độ trễ để tự động dừng hoặc rollback.

Kết quả và minh chứng cần đạt

- Pipeline hoàn chỉnh và lịch sử artifact truy vết được từ commit đến deployment.
- Manifest chiến lược blue-green/canary; bằng chứng phân phối lưu lượng.
- Biểu đồ metric trước - trong - sau release lỗi và bằng chứng rollback tự động.
- Bảng so sánh hai chiến lược về tài nguyên, tốc độ, rủi ro và yêu cầu quan sát.

Tài liệu nghiên cứu gợi ý. [30], [55], [56] và Bài giảng Chương 5.

Công nghệ gợi ý. GitHub Actions/GitLab CI; Kubernetes; Argo Rollouts hoặc Flagger; Prometheus; Grafana; Nginx/Istio; k6.

Đề tài 28 Quản lý hạ tầng bằng mã và GitOps với phát hiện sai lệch cấu hình

Câu hỏi nghiên cứu. Làm thế nào để Git trở thành nguồn sự thật cho hạ tầng và ứng dụng nhưng vẫn kiểm soát state, secret và thay đổi ngoài quy trình?

Mục tiêu của đề tài

- Mô-đun hóa mã hạ tầng và quản lý state an toàn cho nhiều môi trường.
- Triển khai ứng dụng bằng GitOps với cơ chế đồng bộ và quay lui.
- Phát hiện, giải thích và xử lý configuration drift.

Nội dung lý thuyết cần báo cáo

- Declarative IaC, resource graph, state, plan/apply và idempotency.
- Module, environment, remote state, locking và secret trong IaC.
- Pull-based GitOps, reconciliation, desired/live state và drift.
- Change review, policy as code, separation of duties và break-glass change.

Yêu cầu thực nghiệm

- Viết module cấp phát mạng, compute hoặc cụm local; tạo dev và staging từ cấu hình dùng lại.
- Cấu hình CI chạy fmt/validate/plan/security scan và yêu cầu phê duyệt trước apply.
- Dùng Argo CD/Flux đồng bộ manifest ứng dụng; thay đổi trực tiếp live state để tạo drift.
- Quan sát phát hiện/tự phục hồi drift; thử rollback bằng Git revert và xử lý một thay đổi break-glass.

Kết quả và minh chứng cần đạt

- Kho IaC có module, tài liệu biến đầu vào/đầu ra và state không nằm trong Git.
- Pipeline kiểm tra plan và policy; GitOps dashboard thể hiện trạng thái đồng bộ.
- Biên bản thí nghiệm drift gồm thời gian phát hiện, hành vi tự sửa và ngoại lệ.
- Quy trình thay đổi chuẩn và break-glass, kèm kiểm toán sau thay đổi.

Tài liệu nghiên cứu gợi ý. [10], [29], [30] và Bài giảng Chương 5.

Công nghệ gợi ý. Terraform hoặc OpenTofu; Terratest tùy chọn; Argo CD/Flux; Kubernetes; Checkov/Conftest; SOPS/Sealed Secrets.

Đề tài 29 Xây dựng khả năng quan sát và SRE gắn với tối ưu chi phí cho ứng dụng microservices

Câu hỏi nghiên cứu. Làm thế nào liên kết log, metric, trace, SLO và chi phí để ưu tiên đúng cải tiến vận hành thay vì chỉ tạo nhiều dashboard?

Mục tiêu của đề tài

- Instrument một ứng dụng nhiều dịch vụ bằng telemetry có correlation ID.
- Định nghĩa SLI/SLO và cảnh báo theo triệu chứng người dùng.
- Tính chi phí trên một đơn vị giao dịch và tìm cơ hội rightsizing không vi phạm SLO.

Nội dung lý thuyết cần báo cáo

- Ba tín hiệu telemetry; distributed trace, context propagation và sampling.

- Golden signals, RED/USE, SLI, SLO, error budget và burn-rate alert.
- MTTR, runbook, toil và post-incident review.
- FinOps, allocation/tagging, unit economics, rightsizing và chi phí nhàn rỗi.

Yêu cầu thực nghiệm

- Triển khai ứng dụng 2-3 dịch vụ; thu log, metric và trace qua OpenTelemetry Collector.
- Định nghĩa SLO độ sẵn sàng/độ trễ; tạo cảnh báo burn rate và runbook.
- Gây lỗi chậm ở một dịch vụ, dùng trace để xác định nút thắt và đo thời gian chẩn đoán.
- Thu thập mức dùng CPU/RAM, ước tính chi phí theo request; rightsizing rồi chạy lại tải để kiểm tra SLO.

Kết quả và minh chứng cần đạt

- Dashboard liên kết metric với trace và log; truy vết được một yêu cầu qua các dịch vụ.
- Tài liệu SLI/SLO, error budget, cảnh báo và runbook có kiểm thử.
- Báo cáo sự cố giả lập với timeline, nguyên nhân và thời gian chẩn đoán.
- Bảng chi phí/request trước - sau rightsizing và bằng chứng SLO vẫn được duy trì.

Tài liệu nghiên cứu gợi ý. [13]-[15], [31] và Bài giảng Chương 5.

Công nghệ gợi ý. OpenTelemetry; Prometheus; Grafana; Tempo/Jaeger; Loki/OpenSearch; Kubernetes; Kubecost hoặc OpenCost; k6.

Đề tài 30 Triển khai dịch vụ suy luận AI trên đám mây với tự động co giãn và giám sát

Câu hỏi nghiên cứu. Một mô hình máy học nhỏ cần được đóng gói, phục vụ và giám sát thế nào để cân bằng độ trễ, thông lượng, chi phí và rủi ro dữ liệu?

Mục tiêu của đề tài

- Đóng gói mô hình đã huấn luyện thành API suy luận có version và schema đầu vào.
- Triển khai trên container hoặc serverless, cấu hình co giãn và đo cold start.
- Theo dõi hiệu năng hệ thống, chất lượng đầu ra cơ bản và chi phí trên 1.000 lượt suy luận.

Nội dung lý thuyết cần báo cáo

- Vòng đời MLOps, model registry, versioning, reproducibility và serving.
- Online versus batch inference; CPU/GPU, batching, concurrency và model loading.
- Latency, throughput, cold start, drift, data quality và model monitoring.
- Bảo mật endpoint, dữ liệu đầu vào, model artifact, rate limit và AI risk ở mức phù hợp.

Yêu cầu thực nghiệm

- Chọn mô hình công khai nhỏ cho phân loại văn bản/ảnh; đăng ký model và tạo image/API có health endpoint.
- Triển khai hai cấu hình tài nguyên hoặc hai cách phục vụ; tạo tải theo batch size/concurrency khác nhau.
- Đo p50/p95 latency, throughput, memory, thời gian khởi động, tỷ lệ lỗi và chi phí ước tính.
- Bổ sung xác thực/rate limit, log không chứa dữ liệu nhạy cảm và cảnh báo thay đổi phân bố đầu vào mô phỏng.

Kết quả và minh chứng cần đạt

- Model artifact có version, dependency và input signature; API được đóng gói tái lập.
- Dashboard hạ tầng và ứng dụng; bảng benchmark theo cấu hình/batch/concurrency.
- Ước tính chi phí cho 1.000 và 1 triệu lượt suy luận, nêu điểm hòa vốn giữa hai phương án.
- Model card ngắn, phân tích dữ liệu/rủi ro và giới hạn không được suy diễn quá kết quả thử nghiệm.

Tài liệu nghiên cứu gợi ý. [26], [27], [57], [58] và Bài giảng Chương 5.

Công nghệ gợi ý. scikit-learn/ONNX hoặc mô hình Hugging Face nhỏ; MLflow; FastAPI; Docker; Kubernetes/Knative hoặc Cloud Run; Prometheus/OpenTelemetry; Locust/k6.

Gợi ý tiêu chí đánh giá đồ án

Bảng dưới đây là khung tham khảo dùng chung. Giảng viên có thể thay đổi trọng số cho từng đề tài, dựa trên tỷ trọng của thực nghiệm và phân tích kết quả, vì đây là phần phân biệt đồ án nghiên cứu với báo cáo cài đặt công cụ.

Thành phần	Điểm	Minh chứng đánh giá
Cơ sở lý thuyết và câu hỏi nghiên cứu	15	Thuật ngữ đúng; có mô hình/giả thuyết; tài liệu chính thống; phạm vi rõ.
Thiết kế kiến trúc và phương pháp	15	Yêu cầu truy vết được; thiết kế có đánh đổi; thí nghiệm kiểm soát biến và có baseline.
Mức hoàn thiện thực nghiệm	25	Hệ thống chạy được; có kịch bản bình thường và lỗi; cấu hình có thể tái lập.
Dữ liệu đo và phân tích	20	Dữ liệu thô; số lần lặp; biểu đồ đúng; diễn giải nguyên nhân, sai số và giới hạn.
An toàn bảo mật và quản trị chi phí	10	Không lộ secret; quyền tối thiểu; dọn tài nguyên; ước tính hoặc kiểm soát chi phí.
Mã nguồn tài liệu và khả năng tái lập	10	README, version, script/IaC, test, cấu trúc kho và lịch sử đóng góp rõ.
Báo cáo trình bày và phản biện	5	Báo cáo mạch lạc; demo đúng trọng tâm; cả ba thành viên trả lời được quyết định kỹ thuật.

Tài liệu tham khảo

Danh mục dưới đây ưu tiên tiêu chuẩn, tài liệu chính thức của dự án mã nguồn mở và tài liệu nhà cung cấp. Khi thực hiện đồ án, sinh viên cần ghi phiên bản phần mềm và ngày truy cập đối với tài liệu trực tuyến vì giao diện và tính năng dịch vụ đám mây có thể thay đổi.

[1] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” NIST SP 800-145, 2011. https://doi.org/10.6028/NIST.SP.800-145

[2] F. Liu et al., “NIST Cloud Computing Reference Architecture,” NIST SP 500-292, 2011. https://doi.org/10.6028/NIST.SP.500-292

[3] NIST, “Evaluation of Cloud Computing Services Based on NIST SP 800-145,” NIST SP 500-322, 2018. https://doi.org/10.6028/NIST.SP.500-322

[4] L. Q. Tuấn, Bài giảng Điện toán đám mây Chương 1 Tổng quan điện toán đám mây, 2026.

[5] L. Q. Tuấn, Bài giảng Điện toán đám mây Chương 2 đến Chương 5, 2026.

[6] Amazon Web Services, “AWS Well-Architected Framework.” https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html

[7] Microsoft, “Azure Well-Architected Framework.” https://learn.microsoft.com/azure/well-architected/

[8] Google Cloud, "Google Cloud Well-Architected Framework." https://cloud.google.com/architecture/framework

[9] Kubernetes, "Kubernetes Documentation." https://kubernetes.io/docs/

[10] HashiCorp, "Terraform Documentation." https://developer.hashicorp.com/terraform/docs

[11] Docker, "Docker Documentation." https://docs.docker.com/

[12] Cloud Native Computing Foundation, "Cloud Native Definition." https://github.com/cncf/toc/blob/main/DEFINITION.md

[13] OpenTelemetry, "Documentation." https://opentelemetry.io/docs/

[14] Prometheus Authors, "Prometheus Documentation." https://prometheus.io/docs/

[15] FinOps Foundation, "FinOps Framework." https://www.finops.org/framework/

[16] S. Rose et al., "Zero Trust Architecture," NIST SP 800-207, 2020. https://doi.org/10.6028/NIST.SP.800-207

[17] R. Chandramouli and Z. Butcher, "A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments," NIST SP 800-207A, 2023. https://doi.org/10.6028/NIST.SP.800-207A

[18] Cloud Security Alliance, "Cloud Controls Matrix." https://cloudsecurityalliance.org/research/cloud-controls-matrix

[19] OWASP Foundation, "OWASP Kubernetes Top Ten." https://kubernetes-top10.owasp.org/

[20] OWASP Foundation, "Serverless FaaS Security Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/Serverless_FaaS_Security_Cheat_Sheet.html

[21] NIST, "Incident Response Recommendations and Considerations for Cybersecurity Risk Management," NIST SP 800-61 Rev. 3, 2025. https://doi.org/10.6028/NIST.SP.800-61r3

[22] CIS, "CIS Benchmarks." https://www.cisecurity.org/cis-benchmarks

[23] OWASP Foundation, "OWASP Cloud-Native Application Security Top 10." https://owasp.org/www-project-cloud-native-application-security-top-10/

[24] NIST, "Contingency Planning Guide for Federal Information Systems," NIST SP 800-34 Rev. 1, 2010. https://doi.org/10.6028/NIST.SP.800-34r1

[25] Apache Spark, "Structured Streaming Programming Guide." https://spark.apache.org/docs/latest/streaming/getting-started.html

[26] MLflow, "MLflow Models." https://mlflow.org/docs/latest/ml/model/

[27] MLflow, "MLflow Model Serving." https://mlflow.org/docs/latest/ml/deployment/

[28] OpenFaaS, "OpenFaaS Documentation." https://docs.openfaas.com/

[29] OpenGitOps, "OpenGitOps Principles." https://opengitops.dev/

[30] Argo Project, "Argo CD Documentation." https://argo-cd.readthedocs.io/

[31] Kubernetes, "Autoscaling Workloads." https://kubernetes.io/docs/concepts/workloads/autoscaling/

[32] The Twelve-Factor App. https://12factor.net/

[33] Proxmox, “Proxmox VE Administration Guide.” https://pve.proxmox.com/pve-docs/pve-admin-guide.html

[34] Knative, “Knative Documentation.” https://knative.dev/docs/

[35] MinIO, “MinIO Object Store Documentation.” https://min.io/docs/minio/linux/index.html

[36] Linux Foundation, “NFS Documentation.” https://docs.kernel.org/filesystems/nfs/index.html

[37] J. Axboe, “fio Documentation.” https://fio.readthedocs.io/

[38] PostgreSQL Global Development Group, “PostgreSQL Documentation.” https://www.postgresql.org/docs/

[39] MongoDB, “MongoDB Manual.” https://www.mongodb.com/docs/manual/

[40] Apache Cassandra, “Apache Cassandra Documentation.” https://cassandra.apache.org/doc/latest/

[41] Apache Iceberg, “Apache Iceberg Documentation.” https://iceberg.apache.org/docs/latest/

[42] Linux Foundation, “Delta Lake Documentation.” https://docs.delta.io/

[43] Apache Kafka, “Apache Kafka Documentation.” https://kafka.apache.org/documentation/

[44] Apache Flink, “Apache Flink Documentation.” https://flink.apache.org/what-is-flink/flink-architecture/

[45] OpenMetadata, “OpenMetadata Documentation.” https://docs.open-metadata.org/

[46] Keycloak, “Keycloak Documentation.” https://www.keycloak.org/documentation

[47] HashiCorp, “Vault Documentation.” https://developer.hashicorp.com/vault/docs

[48] OWASP Foundation, “Secrets Management Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

[49] SLSA, “Supply-chain Levels for Software Artifacts.” https://slsa.dev/

[50] CycloneDX, “CycloneDX Bill of Materials Standard.” https://cyclonedx.org/

[51] Wazuh, “Wazuh Documentation.” https://documentation.wazuh.com/

[52] Velero, “Velero Documentation.” https://velero.io/docs/

[53] Microsoft, “Cloud Design Patterns.” https://learn.microsoft.com/azure/architecture/patterns/

[54] Sigstore, “Cosign Documentation.” https://docs.sigstore.dev/cosign/

[55] Argo Project, “Argo Rollouts Documentation.” https://argo-rollouts.readthedocs.io/

[56] GitHub, “GitHub Actions Documentation.” https://docs.github.com/actions

[57] NIST, “Artificial Intelligence Risk Management Framework,” NIST AI 100-1, 2023. https://doi.org/10.6028/NIST.AI.100-1

[58] Cloud Security Alliance, “AI Controls Matrix.” https://cloudsecurityalliance.org/artifacts/ai-controls-matrix-v1-1